

Group Assignment [10%]

DUE DATE: 24th May 2026, 11.59 PM

Objective

You need to design and implement a Java program with a simple window interface (buttons, text boxes, etc.) that generates an encoded version of a user-provided string. The encoding process will use a unique, hidden group-specific shift key embedded within the code. This ensures that the encoded output is distinct and traceable to each student group, even for identical input strings.

Class Requirements

You need to create a class called **Encoded** that follows the design shown in the UML diagram. The diagram lists the minimum required data fields and methods your class must have. You are free to add helper methods or fields as needed for clarity and modularity.

class Encoded	
-	inputText :String
-	charCount :int
-	resultText :String
-	groupID :String // Secret key embedded in code
+	Encoded()
+	Encoded(inputText: String)
+	countCharacters(inputText: String): int
+	checkStringValidity(inputText: String): boolean
+	generateShift(): int
+	applyCipher(inputText: String, shift: int): String

Program Input

The program prompts for a String literal (of any random length) and echoes an encoded String literal. The user is prompted to enter:

- A string to encode via GUI
 - The string must contain only alphanumeric (Lowercase letters) and spaces.
 - If the input is invalid, display an error and prompt again.
- The Group ID is not entered by the user — it is internally defined in the code and must not be visible or changeable at runtime.

Encoding Rules

1. Validate input: Accept only alphanumeric (Lowercase letters) and whitespace (ws) using `checkStringValidity()`.

2. Count the number of non-space characters in the input using `countCharacters()`.
3. Generate `groupShift` using `hashCode()` from a hardcoded secret group ID (for example `groupID = G08/NC-G18`). The generated `groupShift` value must be between 1 and 10 and will always be the same.
4. Final shift for encoding the **inputText** is calculated as:

$$\text{finalShift} = \text{groupShift} + \text{number of non-space characters}.$$
5. Encode characters:
 - For lowercase letters a–z:

$$(c - 'a' + \text{finalShift}) \% 26 + 'a'$$
 - For digits 0–9:

$$(c - '0' + \text{finalShift}) \% 10 + '0'$$
 - Spaces remain unchanged

Example

Group ID that is hardcoded should be in this format `LectureGroupNumber/Programme-GroupNumber` (for example: `G08/NC-G18`). Follow the group name listed in the group registration. Example of the final encoding shift rule is as below:

```
private final String groupID = "G08/NC-G18";
```

Input: "fcsit 678"

- Non-space characters: 8
- `groupShift` = 7 (from hash of `G08/NC-G18`)
- Final shift: $7 + 8 = 15$

Output: "urhxi 123"

GUI Requirements

Use Java Swing

- Input text field
- Encode button
- Display area for non-space characters, final shift and encoded result

NOTE: You may optionally show "Encoding completed successfully" messages, but do NOT display group ID or `groupShift`.

Code Documentation and Contribution Attribution

All code must be thoroughly commented to explain the logic and functionality of each section. Comments should clearly describe the purpose of methods, the rationale behind key computations (e.g., shifting algorithm), and any input/output expectations. Additionally, each group member must indicate the specific portions of the code they were responsible for, either

through inline comments or by tagging method blocks (e.g., `// Contributed by [Member Name]`). This ensures transparency of individual contributions and facilitates fair evaluation.

Submission Instructions:

1. Name your main class file as **Assignment.java**. This is the file that we will look for to compile and execute. **If you don't have this file, you will get 0.**
2. You would also need to hand in the **faculty assignment cover page (.pdf only)** with the signature of each member agreeing to the terms written on the cover page.
3. **Report Submission (Maximum 2 Pages, Excluding Cover Page)** – a concise report with a maximum length of **two (2) pages**. The report must include the following components:
 - a) **Code Contribution Table**
Provide a clearly structured table indicating the distribution of class and method implementation responsibilities among group members. Each member's contribution should be mapped to specific parts of the codebase (e.g., class names, methods, UI components).
 - b) **Group Shift Explanation**
State your **groupShift value** and explain how it was derived from the embedded group ID in your code. Include a brief description of the hashing mechanism and how it contributes to the uniqueness of your encoded output.
4. The java code files should contain comments with explanations and indicate which part of the code a specific member is contributing.
5. **Zip (not .rar)** your .java file(s), cover page, and group discussion screenshots with the following naming convention, **groupID.zip**. For example, **G08_NC-G18.zip**.
6. Submit your .zip file through the link on the assignment submission page.
7. Fill in the peer evaluation form (will be activated before the due date).
8. Late submission will result in a penalty. If your group member did not contribute to the assignment at all, he/she will get 0 marks for the entire assignment.

Marking Rubric

Criteria	CLO	0	1	2	3	4
Appropriate Use of Java Tools & Libraries (e.g., Swing Hashing, Data Types)	CLO1	No understanding of Java tools; irrelevant or incorrect tools used.	Very limited understanding; tools misused and poorly applied.	Basic tool usage; some elements are functional but lack efficiency.	Good selection of tools; mostly used correctly and appropriately.	
Object-Oriented Application Design (e.g., class structure, encapsulation, methods)	CLO2	No structure or OOP principles implemented.	Limited structure; very minimal application of OOP concepts.	Basic class and method setup; some use of OOP principles.	Functional structure with fair implementation of OOP practices.	Excellent modular OOP design; strong encapsulation and method structure.
Teamwork & Peer Evaluation (e.g., comments, contributor tags, collaboration)	CLO3	No indication of teamwork. Peer evaluation not submitted or shows no engagement.	Some team contributions noted. Peer evaluation shows imbalance in effort or partial collaboration.	Most code sections show clear team roles. Peer evaluation shows moderate but uneven participation.	Contributors are clearly identified. Peer evaluation supports fair collaboration.	